

CMS News Portal

A role-based news / content management website: readers register, write articles and comment; admins moderate everything from a dashboard. Built with core PHP and MySQL on XAMPP.

PHP 8.2

MySQL / MariaDB

Apache (XAMPP)

HTML · CSS · JS

Session Auth

3 Roles

01 Tech Stack

FRONTEND

- **HTML5** — page structure
- **CSS3** — custom styling, CSS variables, responsive grid
- **Vanilla JavaScript** — filters, form validation, newsletter
- Self-hosted fonts (works offline): Inter + Playfair Display

BACKEND

- **PHP** (procedural) — server logic
- **mysqli** — database driver
- **Sessions** — login state & roles
- Front-controller router in `index.php`

DATA & SERVER

- **MySQL / MariaDB** — database `cms`
- **Apache** via XAMPP (port `8080`)
- 4 tables: users, posts, comments, subscribers
- Passwords hashed with `bcrypt`

02 How It Works — Architecture

REQUEST FLOW

Browser → `index.php?page=home` → router → `home.php`

- `index.php` is the single entry point (**front controller**)
- It starts the session, opens the DB connection `$conn`, reads `?page=`
- A `switch` statement `require`s the matching page file
- Every protected page re-checks the session role at the top

DATABASE — RELATIONSHIPS

TABLE	HOLDS	LINKS
users	accounts + role	—
posts	articles	user_id → users
comments	comments	post_id, user_id

- One user → many posts → many comments
- Foreign keys use `ON DELETE CASCADE`

03 Features to Demo · all tested working

PUBLIC (NO LOGIN)

- Browse latest articles + real cover photos
- Filter by category (12 categories)
- Read full article + comments
- Trending list, live site stats
- Register / Login

READER (LOGGED IN)

- Write an article → saved as **Pending**
- My Posts & My Comments history
- Comment on published articles
- Edit profile (needs current password)

ADMIN

- Dashboard with live counts
- Manage Posts — search + status/category filters
- Publish / unpublish / delete posts
- Manage Comments — delete
- Manage Users — edit details & role

04 Strong Points & Weak Points

✓ STRONG POINTS

- **Front-controller routing** — one clean entry point
- **Hashed passwords** — never stored as plain text
- **Role-based access control** guards every admin page
- **Moderation workflow** — posts stay pending until approved
- **Prepared statements** used on most write actions
- **Output escaping** (`htmlspecialchars`) blocks XSS
- Live database-driven stats & filters (nothing hardcoded)
- Responsive, consistent UI; works offline

! WEAK POINTS (BE HONEST)

- **Inconsistent DB access** — some queries escape input instead of using prepared statements
- **No CSRF protection** on forms & actions
- Admin actions run through **GET links** (delete via URL)
- DB credentials hardcoded (`root` , no password) — local only
- Uploaded images aren't validated (type/size)
- `content_creator` / `superadmin` roles exist but are unused
- No real pagination on the posts list

05 Likely Questions & Answers

Q1 What is the job of `index.php` ?

It's the **front controller / router**. Every request goes through it. It starts the session, opens the database connection, reads the `?page=` value, and includes the correct page file with a `switch` statement.

Q2 How is login and authentication handled?

Session-based. On login we look up the user, verify the password with `password_verify()`, then store `user_id`, `username` and `role` in `$_SESSION`. The role decides whether you land on the dashboard or the home page.

Q3 How are passwords stored securely?

They are hashed with PHP's `password_hash()` (bcrypt) before saving, and checked with `password_verify()`. The real password is never stored or compared directly.

Q4 How do you prevent SQL injection?

Mainly with **prepared statements** (`bind_param`) for inserts and updates; other inputs are escaped or cast to integers. (Honest note: a few older queries still use manual escaping — making everything prepared would be the improvement.)

Q5 How do you prevent XSS (cross-site scripting)?

All user text is escaped with `htmlspecialchars()` when printed back to the page, so any HTML/script a user types is shown as plain text, not executed.

Q6 What are the user roles and what can each do?

Reader (default) — write articles (pending) and comment. **Admin / superadmin** — moderate posts & comments and manage users. The role is stored in the DB and checked on every protected page.

Q7 Explain the article publishing workflow.

A reader submits an article → it is saved with status `pending` → an admin reviews it in Manage Posts and clicks approve → status becomes `published` and it appears on the site.

Q8 Describe the database design.

Three main tables — `users`, `posts`, `comments`. A user has many posts; a post has many comments. Foreign keys link them with `ON DELETE CASCADE`, so deleting a user or post cleans up related rows.

Q9 How does the category filter work?

On the home page each category pill links to `?category=X` and the query filters with a prepared statement. On the admin Manage Posts page the search + filters are done live in the browser with JavaScript.

Q10 Why mysqli and not PDO? How would you improve the project?

`mysqli` was chosen for simplicity (both work with MySQL). Improvements: use prepared statements everywhere, add CSRF tokens, move DB credentials to a config/env file, validate uploads, and add HTTPS for production.

06 Demo Cheat-Sheet

SITE URL

`localhost:8080/de6/`

ADMIN LOGIN

user: `admin`

SAMPLE READERS

mustufa / meet / yusuf · pass `123`

TEAM

Mustufa · Meet · Yusuf

ARTICLES

44

CATEGORIES

12

TABLES

4

ROLES

3

Before the viva: start **Apache** and **MySQL** in the XAMPP control panel, then open the site URL. Keep phpMyAdmin open in another tab to show the tables if asked.